Reports On Main Project

Name : Nithya S

Batch code : TN_DA_FNB10

Mail id : nithyasubburaj2@gmail.com

Contact no : +91 9384760769

Project Title : End-to-End Sales Performance Analysis of Retail Store Transaction Data using Python and Power BI

Project Domain : Retail / E-commerce Sales

Tools Used : Python, Power BI

Submission Date : 17/05/2026

Mentor Name : Kumaran M

Raw Dataset Link : Raw -Retail Store Sales Dataset

Cleaned Dataset Link: Cleaned Retail Store Sales Dataset

Google Colab Link : DA Final Project Retail Store Data

End-to-End Sales Performance Analysis of Retail Store Transaction Data using Python and Power BI

1. Overview:

This project presents an end-to-end sales performance analysis of retail store transaction data using Python and Power BI. The dataset was cleaned, transformed, and analyzed to understand sales trends, customer behaviour, and business performance. Various exploratory data analysis techniques and interactive dashboard visualizations were used to generate meaningful insights and support data-driven decision-making.

2. Objective:

- Analyze retail store transaction data to understand overall sales performance using exploratory data analysis (EDA)
- Identify top-performing and low-performing products/categories based on revenue and quantity sold.
- Evaluate customer purchasing behaviour and spending patterns
- Identify monthly/seasonal sales trends using transaction dates.
- Detect data quality issues and perform data cleaning, preprocessing and transformation for accurate analysis.
- Build meaningful visualizations and dashboards to communicate business insights.
- Generate actionable recommendations to improve revenue, customer satisfaction, and business performance.

3. Purpose Of the Project:

The purpose of this project is to demonstrate how raw retail transaction data can be transformed into meaningful business insights using data analytics techniques. The project aims to help businesses understand customer behaviour, monitor sales trends, optimize decision-making, and improve overall business performance through data-driven strategies. This project also serves as a real-world application of data analytics concepts learned during the course.

4. Dataset Information:

Source: Raw Dataset Retail_store_sales.csv

Domain: Retail/E-commerce

Year / Timeline: Data collected during 2022Jan to 2025Jan

Dataset Column/features Description:

- Transaction ID- A unique identifier for each transaction. Always Not Null and unique
- Customer ID - A unique identifier for each customer.25 unique customers.
- Category - The Product category of the purchased item.
- Item - The name of the purchased item. May contain missing values or None.
- Price Per Unit - The static price of a single unit of the item. May contain missing or None values.
- Quantity - The quantity of the item purchased. May contain missing or None values
- Total Spent - The total amount spent on the transaction. Calculated as Quantity * Price Per Unit.
- Payment Method - The method of payment used. May contain missing or invalid values
- Location - The location where the transaction occurred. May contain missing or invalid values.
- Transaction Date - The date of the transaction. Always Not Null and valid.
- Discount Applied - Indicates if a discount was applied to the transaction. May contain missing values.

5. Tools Used:

Python

Python used for:

- Data cleaning and preprocessing
- Handling missing values
- Removing duplicates

- Formatting columns (Currency, Percentage, Date)
- Outliers & skewness detection
- Feature transformations

Python helped in preparing structured and clean data before importing into Power BI.

Power BI

Power BI user for:

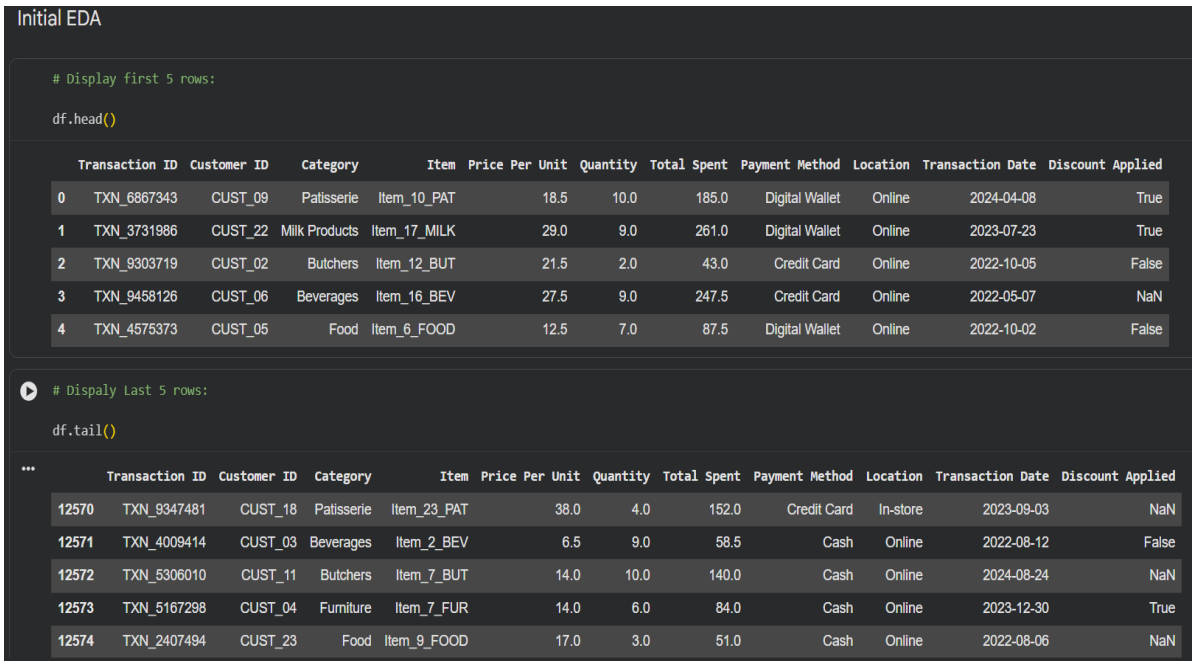
- Creating measures using DAX
- Developing KPIs (Revenue, Cost, Profit, ROAS, ROI, CTR)
- Creating interactive dashboards
- Visualizing trends using charts (Line, Bar, Column, Donut, Treemap)
- Implementing slicers for interactivity

Power BI transformed raw data into meaningful visual insights for business analysis.

6. Initial EDA:

To understand the data structure the basic initial EDAs were performed. Ex: head, info, describe, shape, null checks, duplicate check.

Sample Screenshot:



Initial EDA

```
# Display first 5 rows:

df.head()
```

	Transaction ID	Customer ID	Category	Item	Price Per Unit	Quantity	Total Spent	Payment Method	Location	Transaction Date	Discount Applied
0	TXN_6867343	CUST_09	Patisserie	Item_10_PAT	18.5	10.0	185.0	Digital Wallet	Online	2024-04-08	True
1	TXN_3731986	CUST_22	Milk Products	Item_17_MILK	29.0	9.0	261.0	Digital Wallet	Online	2023-07-23	True
2	TXN_9303719	CUST_02	Butchers	Item_12_BUT	21.5	2.0	43.0	Credit Card	Online	2022-10-05	False
3	TXN_9458126	CUST_06	Beverages	Item_16_BEV	27.5	9.0	247.5	Credit Card	Online	2022-05-07	NaN
4	TXN_4575373	CUST_05	Food	Item_6_FOOD	12.5	7.0	87.5	Digital Wallet	Online	2022-10-02	False

```
# Dispaly Last 5 rows:

df.tail()
```

	Transaction ID	Customer ID	Category	Item	Price Per Unit	Quantity	Total Spent	Payment Method	Location	Transaction Date	Discount Applied
12570	TXN_9347481	CUST_18	Patisserie	Item_23_PAT	38.0	4.0	152.0	Credit Card	In-store	2023-09-03	NaN
12571	TXN_4009414	CUST_03	Beverages	Item_2_BEV	6.5	9.0	58.5	Cash	Online	2022-08-12	False
12572	TXN_5306010	CUST_11	Butchers	Item_7_BUT	14.0	10.0	140.0	Cash	Online	2024-08-24	NaN
12573	TXN_5167298	CUST_04	Furniture	Item_7_FUR	14.0	6.0	84.0	Cash	Online	2023-12-30	True
12574	TXN_2407494	CUST_23	Food	Item_9_FOOD	17.0	3.0	51.0	Cash	Online	2022-08-06	NaN

```
#Check duplicates:

df.duplicated().sum()

np.int64(0)
```

```
#Check data types:

df.dtypes
```

	0
Transaction ID	object
Customer ID	object
Category	object
Item	object
Price Per Unit	float64
Quantity	float64
Total Spent	float64
Payment Method	object
Location	object
Transaction Date	object
Discount Applied	object

dtype: object

```
# Dataset information:

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12575 entries, 0 to 12574
Data columns (total 11 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Transaction ID    12575 non-null  object
 1   Customer ID       12575 non-null  object
 2   Category          12575 non-null  object
 3   Item              11362 non-null  object
 4   Price Per Unit    11966 non-null  float64
 5   Quantity          11971 non-null  float64
 6   Total Spent       11971 non-null  float64
 7   Payment Method    12575 non-null  object
 8   Location          12575 non-null  object
 9   Transaction Date  12575 non-null  object
 10  Discount Applied  8376 non-null   object
dtypes: float64(3), object(8)
memory usage: 1.1+ MB
```

```
# Dataset Shape:

df.shape
```

```
(12575, 11)
```

7. Data Cleaning & Pre-processing:

In this step, Raw data were cleaned by remove duplicates, correct data types, handling missing values and checked and treated outliers by using python.

Sample Screenshot of data cleaning:

```
# Datatype Changes

df["Transaction Date"] = pd.to_datetime(df["Transaction Date"])

df["Quantity"] = df["Quantity"].fillna(0).astype(int)

df['Discount Applied'] = df['Discount Applied'].astype(bool)

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12575 entries, 0 to 12574
Data columns (total 11 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Transaction ID    12575 non-null  object
 1   Customer ID       12575 non-null  object
 2   Category          12575 non-null  object
 3   Item              11362 non-null  object
 4   Price Per Unit    11966 non-null  float64
 5   Quantity          12575 non-null  int64
 6   Total Spent       11971 non-null  float64
 7   Payment Method    12575 non-null  object
 8   Location          12575 non-null  object
 9   Transaction Date  12575 non-null  datetime64[ns]
 10  Discount Applied  12575 non-null  bool
dtypes: bool(1), datetime64[ns](1), float64(2), int64(1), object(6)
memory usage: 994.8+ KB
```

```python
# Check for and drop duplicate rows, keeping the first occurrence
duplicate_count = df.duplicated().sum()
print(f"Number of duplicate rows found: {duplicate_count}")

if duplicate_count > 0:
    df.drop_duplicates(inplace=True)
    print(f"Removed {duplicate_count} duplicate rows. New shape: {df.shape}")
else:
    print("No duplicate rows found.")
```

```
Number of duplicate rows found: 0
No duplicate rows found.
```

Missing values were handled using relationship-based imputation. Missing Item values were inferred using the relationship between Category and Price Per Unit. Missing numerical values (Price Per Unit, Quantity, and Total Spent) were filled using the business rule Total Spent = Price Per Unit × Quantity to maintain data consistency.

1.Fill Discount applied by Mode

```python
# Handling missing value of Discount Applied column
# Finding Mode

df["Discount Applied"].mode()
```

	Discount Applied
0	True

dtype: bool

```python
# Fill the mode value for Discount Applied column

mode_value = df["Discount Applied"].mode()[0]
df["Discount Applied"] = df["Discount Applied"].fillna(value=mode_value)

print(df["Discount Applied"].isnull().sum())
```

```
0
```

2.Fill Price Per Unit by Total spent/Quantity

```
# Handling missing value of Price Per Unit column

df["Price Per Unit"] = df["Price Per Unit"].fillna(
    df["Total Spent"] / df["Quantity"]
)

print(df["Price Per Unit"].isnull().sum())
```

```
0
```

```
# Check Outliers for all numeric column

for col in ["Price Per Unit", "Quantity", "Total Spent"]:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5*IQR
    upper = Q3 + 1.5*IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)]
    print(f"{col}: {len(outliers)} outliers")
```

```
Price Per Unit: 0 outliers
Quantity: 0 outliers
Total Spent: 99 outliers
```

Outliers Found in Total spent column but it is a real business transaction, so keep it because it represents actual customer behaviour. High-value sale is genuine & Important for revenue analysis.

8. Feature Transformation and Metrics Calculations:

Feature transformations were applied to derive meaningful insights from the dataset. Date-based features such as Day of Week, Transaction Month, and Year were extracted from the transaction date. Additional derived fields including Customer Lifetime Value (CLV) and Purchase Frequency were created to analyze customer behavior and purchasing patterns.

```
# Extract time features from the transaction date

df["Year"] = df["Transaction Date"].dt.year
df["Month"] = df["Transaction Date"].dt.month
df["Day_of_week"] = df["Transaction Date"].dt.day_name()

df['Is_weekend'] = df['Transaction Date'].dt.dayofweek.isin([5, 6]).astype(int)

df.head()
```

	Transaction ID	Customer ID	Category	Item	Price Per Unit	Quantity	Total Spent	Payment Method	Location	Transaction Date	Discount Applied	Year	Month	Day_of_week	Is_weekend
0	TXN_6867343	CUST_09	Patisserie	Item_10_PAT	18.5	10	185.0	Digital Wallet	Online	2024-04-08	True	2024	4	Monday	0
1	TXN_3731986	CUST_22	Milk Products	Item_17_MILK	29.0	9	261.0	Digital Wallet	Online	2023-07-23	True	2023	7	Sunday	1

```
# Customer Lifetime Value (CLV)
df["CLV"] = df.groupby("Customer ID")["Total Spent"].transform("sum")

# Purchase Frequency
df["Purchase_Frequency"] = df.groupby("Customer ID")["Transaction ID"].transform("count")

# Display results
print(df[["Customer ID", "Total Spent", "CLV", "Purchase_Frequency"]].head(10))
```

```
  Customer ID  Total Spent      CLV  Purchase_Frequency
0     CUST_09        185.0  61423.5                 519
1     CUST_22        261.0  61732.5                 501
2     CUST_02         43.0  62046.5                 488
3     CUST_06        247.5  58632.5                 481
4     CUST_05         87.5  66974.5                 544
5     CUST_09        200.0  61423.5                 519
6     CUST_07         40.0  60694.5                 491
7     CUST_21          0.0  62933.0                 498
8     CUST_23         27.5  64507.0                 513
9     CUST_25        109.5  57155.5                 476
```

9. EDA and Visualizations:

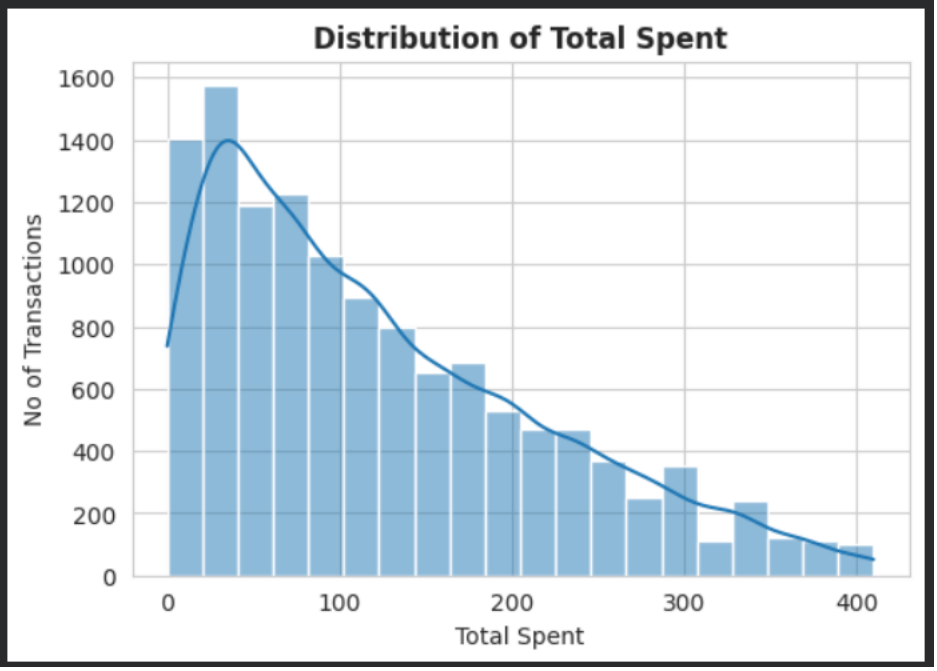
After completing the data cleaning and transformation process in the previous stages, the dataset became ready for Exploratory Data Analysis (EDA). At this stage, the focus is on analyzing patterns, trends, relationships, and business insights from the cleaned dataset.

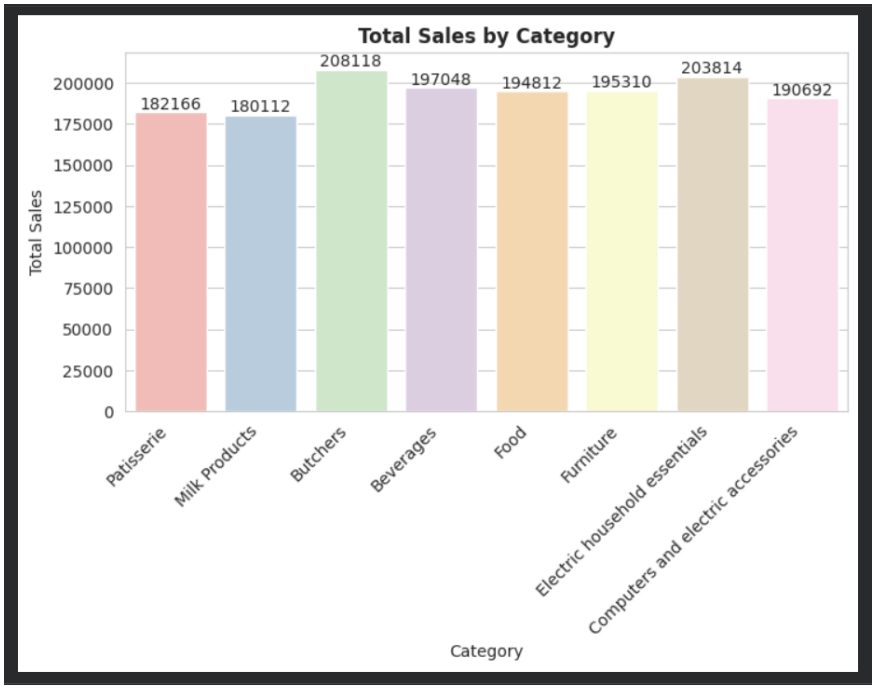
Various univariate, bivariate, and multivariate visualizations are performed to understand customer purchasing behavior, category performance, sales distribution, payment preferences, and revenue trends. Each visualization is interpreted with business-focused insights to explain what the chart represents and how it supports retail business decision-making.

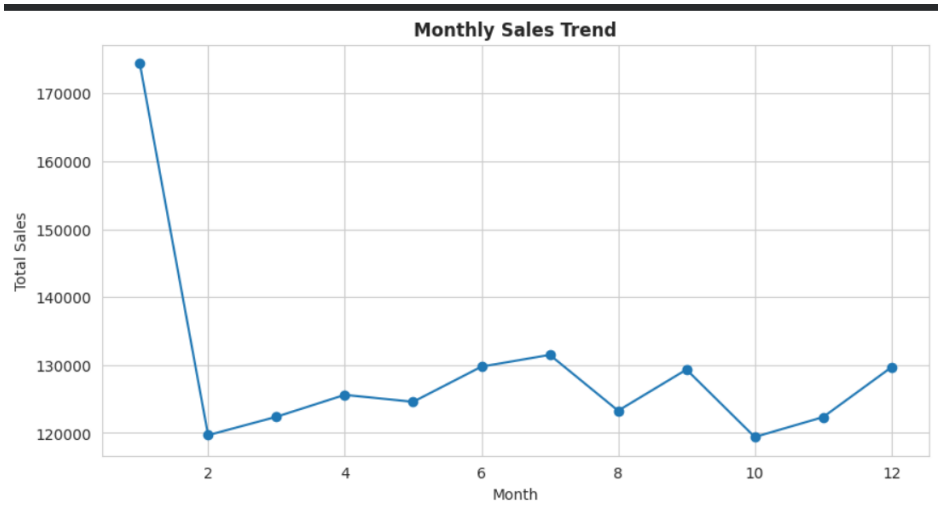
The analysis emphasizes storytelling through data by identifying meaningful patterns and explaining how different features influence sales performance and customer behavior.

Sample Screenshot of Visualization in Python:

```
# Univariate Analysis - Histogram

plt.figure(figsize=(6,4))
sns.histplot(df["Total Spent"], bins=20, kde=True)

plt.title("Distribution of Total Spent",  fontweight="bold")
plt.xlabel("Total Spent")
plt.ylabel("No of Transactions")

plt.show()
```








```
# Multi Variate Analysis - Pivot Table

pivot_table = pd.pivot_table(
    df,
    values="Total Spent",
    index="Category",
    columns="Month",
    aggfunc="sum"
)

pivot_table
```

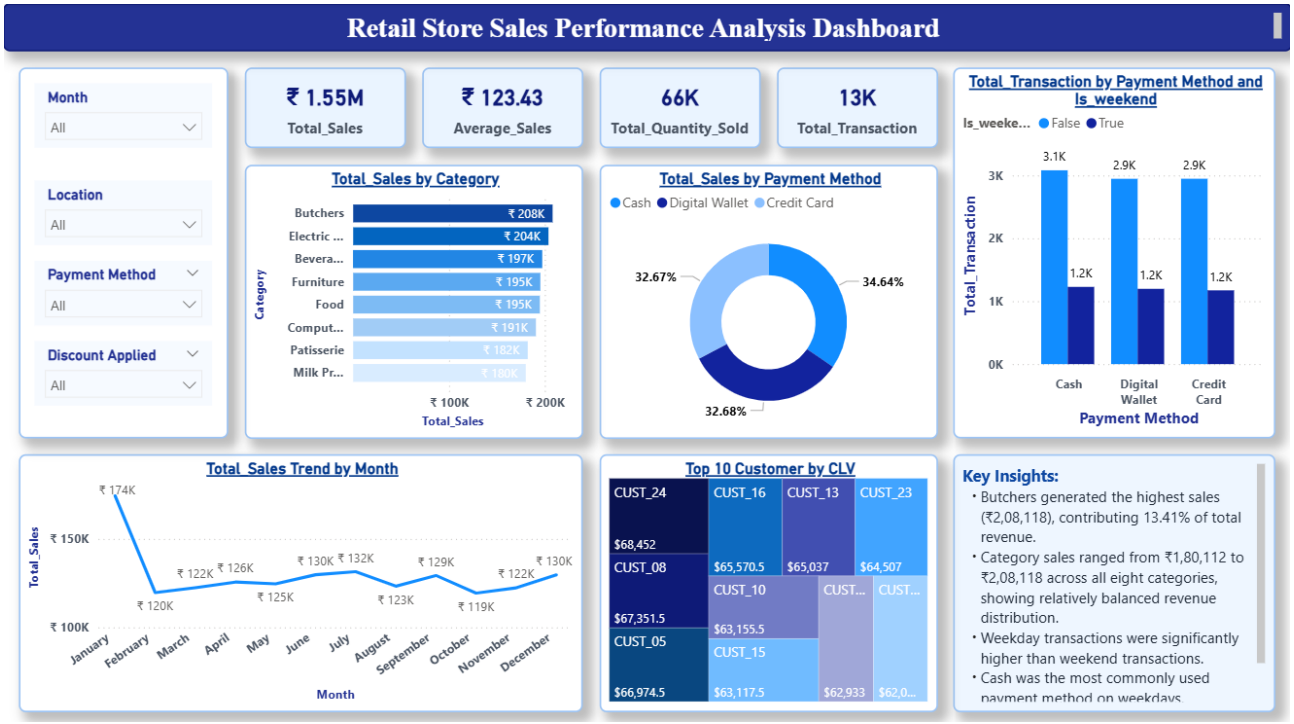
Month	1	2	3	4	5	6	7	8	9	10	11	12
Category												
Beverages	21834.0	17136.5	15127.0	13029.0	17169.0	16198.0	17962.0	15927.5	16592.5	15904.5	15502.0	14665.5
Butchers	24084.5	17832.5	13412.5	16744.0	16552.5	18333.0	17993.5	16598.5	16697.5	18134.0	16040.5	15695.0
Computers and electric accessories	25130.0	12160.5	14687.0	18123.5	12480.5	18860.5	15477.5	13554.0	16201.5	11146.5	15069.5	17801.5
Electric household essentials	21273.0	16964.5	15580.5	17052.0	17238.5	17442.5	17219.0	16517.5	17495.5	12732.5	16842.5	17455.5
Food	22778.5	13788.5	15709.0	12031.5	16021.5	13472.5	19268.5	14352.5	17163.0	18240.0	17649.0	14337.5
Furniture	21312.0	15190.5	16248.0	19786.5	15435.0	15930.0	15169.0	18216.5	14030.0	13050.0	13565.0	17377.5
Milk Products	19588.0	13036.0	15871.5	13247.0	13855.0	12282.0	15383.0	13245.5	16729.5	14993.5	14971.5	16909.5
Patisserie	18421.0	13576.0	15756.5	15605.0	15842.5	17252.5	13036.5	14875.5	14434.5	15212.5	12706.5	15446.5

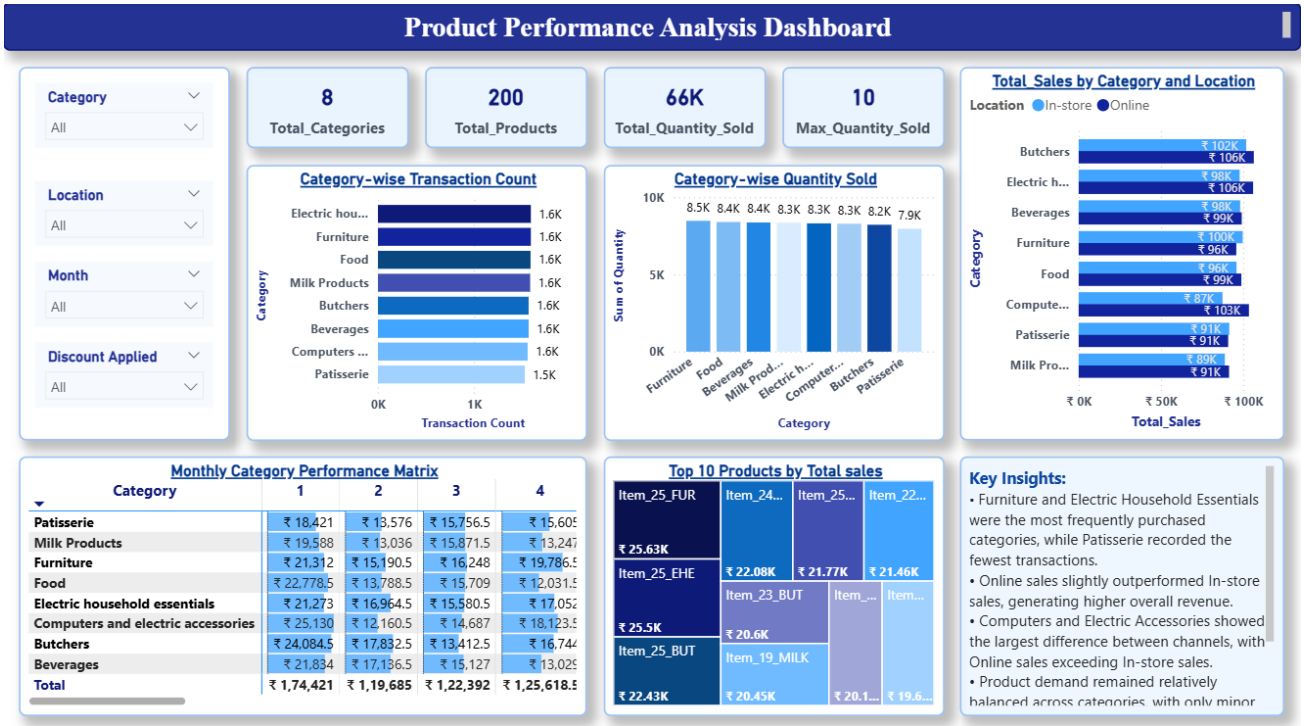
10. Dashboard creation using Power BI:

After the transformation process, Cleaned Data was loaded into Power BI for Visualisation and Interactive Dashboards Creation.

Dashboard:

Sales and Product performance analysis dashboards:





11. Insights & Summary (Top KPIs):

Summary details of the Dashboards:

- Total Sales: ₹1.55M
- Average Transaction Value: ₹123.43
- Total Quantity Sold: 66K
- Total Transactions: 13K

Insights:

- Some categories generate significantly higher sales than others. Categories such as Butchers and Electric Household Essentials are among the major contributors to overall revenue.
- Quantity purchased has a strong positive relationship with Total Spent, indicating that larger purchases directly increase revenue.
- Customers with higher purchase frequency contribute more to Customer Lifetime Value, emphasizing the importance of retaining repeat customers.
- Cash, Credit Card, and Digital Wallet transactions all contribute to sales, showing that customers prefer multiple payment options.
- Sales fluctuate across different months, indicating possible seasonal patterns and changing customer demand throughout the year.

12. Recommendations:

- Increase promotions and maintain sufficient inventory for categories that consistently generate higher revenue.
- Introduce loyalty programs and personalized offers to encourage repeat purchases and increase CLV.
- Use monthly sales trends for demand forecasting and inventory planning.
- Providing flexible payment options improves customer convenience and overall transaction volume.
- Develop personalized campaigns for customers with high CLV to strengthen long-term customer relationships.

13. Conclusion:

This project successfully performed an end-to-end sales performance analysis using Retail Store Transaction Data with Python and Power BI. The raw dataset was cleaned, transformed, and enriched through feature engineering to ensure accurate and reliable analysis. Exploratory Data Analysis (EDA) techniques were applied to uncover patterns, relationships, and trends within the data.

The analysis revealed valuable insights regarding sales performance, customer behavior, purchase frequency, payment preferences, and customer lifetime value. The interactive Power BI dashboard provided a comprehensive view of key business metrics, enabling better understanding of revenue drivers and supporting data-driven decision-making.

Overall, this project demonstrates how data analytics can convert raw transactional data into meaningful business insights. It highlights the practical application of Python for data preparation and Power BI for visualization, showcasing the importance of analytics in improving business performance and strategic planning.

14. Future Enhancement:

- Implement sales forecasting using machine learning models.
- Perform customer segmentation for personalized marketing.
- Build real-time dashboards using live data sources.
- Add geographic analysis for location-wise performance.
- Develop predictive models for customer behavior and sales trends.